Chapter 18 | Worked solutions with Python

Interpreting and Aligning Model Behavior

Work through one score by hand before reading the program. The notebook reads the saved experiment, so it can verify the controls without downloading Qwen.

In [1]

```
from pathlib import Path
import json
import math
import sys

here = Path.cwd().resolve()
root = next(p for p in (here, *here.parents) if
            (p / "examples/ch18_behavior/PRETRAINED_RUN.json").is_file())
if str(root) not in sys.path:
    sys.path.insert(0, str(root))

from examples.ch18_behavior.model import patch_effect, route_policy, summarize

record_path = root / "examples/ch18_behavior/PRETRAINED_RUN.json"
record = json.loads(record_path.read_text())
print(record["model"], "with", len(record["pairs"]), "patching pairs")
```

Output

```
Qwen/Qwen3.5-0.8B-Base with 3 patching pairs
```

1. Recover the odds.

The native update-minus-read gap is $2-4=-2$ and the patched gap is $4-3=1$. The shared softmax denominator cancels in the odds, so the odds of B relative to A change from $e^{-2}=0.1353$ to $e^{1}=2.7183$. If $z_C=6$, then C has the largest logit of the three. Reversing the order of A and B does not establish which of the three is preferred.

In [2]

```
for a, b in ((4, 2), (3, 4)):
    print("gap:", b-a, "odds:", round(math.exp(b-a), 4))
print("top label:", route_policy((3, 4, 6)))
```

Output

```
gap: -2 odds: 0.1353
gap: 1 odds: 2.7183
top label: C
```

2. Measure recovery.

The numerator is $3 - (-2) = 5$. The donor and recipient differ by $2 - (-2) = 4$, so the recovery is $5/4 = 1.25$. The score overshoots the donor. In the second example the numerator still equals 5, but the denominator equals zero. Report the shift of 5 natural-log units and mark the recovery fraction undefined. Native equality does not rule out an effect of the intervention.

In [3]

```
print(patch_effect(-2, 2, 3))
print(patch_effect(-2, -2, 3))
```

Output

```
(5, 1.25)
(5, None)
```

3. Read a feature and change it.

The probe's score changes from 2 to -1. The original model's gap stays at -2 because h_2 stays at 1. For the second intervention hold h_1 at 2 and change h_2 to 0. Then g moves from -2 to 0 while the probe stays at 2. Both comparisons specify the altered coordinate and the downstream outcome.

In [4]

```
for h1, h2 in ((2, 1), (-1, 1), (2, 0)):
    print("vector:", (h1, h2), "probe:", h1, "model gap:", -2*h2)
```

Output

```
vector: (2, 1) probe: 2 model gap: -2
vector: (-1, 1) probe: -1 model gap: -2
vector: (2, 0) probe: 2 model gap: 0
```

4. Check a patch.

A position before the edit has the same causal history in both inputs. Exchanging its vector therefore leaves the score unchanged to numerical precision. After the final block there is no further mixing across token positions. The final normalization and vocabulary readout only use the last vector. Patching any earlier vector at that boundary leaves the gap unchanged. The recorded checks compare both directions.

In [5]

```
blocks = record["pairs"][0]["patches"]
directions = ("read_recipient", "write_recipient")
before = max(abs(layer[key][0]["shift"]) for layer in blocks for key in directions)
earlier_last = max(abs(item["shift"]) for key in directions
                   for item in blocks[-1][key][:-1])
print("largest pre-edit shift:", before)
print("largest earlier-position shift at last block:", earlier_last)
assert max(before, earlier_last) < 0.0006
```

Output

```
largest pre-edit shift: 0.0
largest earlier-position shift at last block: 0.0
```

5. Compare two rewards.

Always reading has four valid labels and two correct proposals per four requests. The other policy has three valid labels and three correct proposals per four requests. A format-only reward prefers the first policy, while the action score prefers the second.

In [6]

```
for proposals in ("AAAA", ("A", "B", "C", "invalid")):
    result = summarize("ABCA", proposals)
    print("format:", result["valid"]/4, "action:", result["correct"]/4)
```

Output

```
format: 1.0 action: 0.5
format: 0.75 action: 0.75
```

6. Apply a threshold.

Subtract 2 and exponentiate to get $(1, e^{-1}, e^{-2})$. Dividing by their sum gives $q(A) = 0.6652$, $q(B) = 0.2447$ and $q(C) = 0.0900$. The constraint alone chooses A. The threshold rule chooses C since $0.6652 < 0.80$. In this case asking delays a request that was already clear.

In [7]

```
logits = (2, 1, 0)
weights = [math.exp(z - max(logits)) for z in logits]
print("restricted shares:", [round(w/sum(weights), 4) for w in weights])
print("decisions:", route_policy(logits), route_policy(logits, 0.8))
```

Output

```
restricted shares: [0.6652, 0.2447, 0.09]
decisions: A C
```

7. Test a shortcut.

For the first pair, compare *I would like to change the saved address* (B) with *Show the saved address before I decide whether to change it* (A). Both contain *change*. For the second pair, compare *Use the new address on my invoice* (B) with *Which address is on file?* (A). Neither contains the cue. Add ambiguous cases with and without the word, for example *I need help with the change form*.

Group templates and their paraphrases by wording family and keep a whole family in one split. Report correct clarification among ambiguous cases and unnecessary clarification among clear cases. Count mistaken update proposals separately so they are visible even when rare.

In [8]

```
targets = ("B", "A", "B", "A", "C")
keyword_policy = ("B", "B", "A", "A", "B")
print(summarize(targets, keyword_policy))
```

Output

```
{'n': 5, 'valid': 5, 'correct': 2, 'right_to_clarify': 0, 'ambiguous': 1, 'incorrect_write': 2}
```

8. Change the decision cost.

The read and update costs stay at 1.05 and 6.00. The new cost of asking is $2(0.60) + 2(0.25) + 0(0.15) = 1.70$, so reading now minimizes expected cost. A real desk would need evidence on the service cost of asking, causing delays and making unintended updates. It would also need calibrated intent estimates for the relevant users and permission rules for any write.

In [9]

```
probabilities = (0.60, 0.25, 0.15)
costs = {"A": (0, 3, 2), "B": (8, 0, 8), "C": (2, 2, 0)}
expected = {action: round(sum(p*c for p, c in zip(probabilities, row, strict=True)), 2)
            for action, row in costs.items()}
print(expected)
print("lowest cost:", min(expected, key=expected.get))
```

Output

```
{'A': 1.05, 'B': 6.0, 'C': 1.7}
lowest cost: A
```